République Tunisienne
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique Université de Sousse

Institut Supérieur des Sciences Appliquées et de Technologie de Sousse

Departement Informatique

RAPPORT DE MINI PROJET WEB

FIA3

Tchanchinet

Élaboré par :

Nermine Galai
Sarra Gharbi

A.U. : 2026/2027

Plan

1. Introduction **3**

2. Cahier des charges / Objectifs **3**

3. Conception **3**

4. Implémentation (Code et explications) **5**

5. Résultat final **13**

6. Difficultés rencontrées et solutions **21**

7. Conclusion **21**

Références **21**

1. Introduction

Dans le cadre de notre mini-projet de développement web, nous avons décidé de developper un quiz inspiré des devinettes tunisiennes présentées dans l'ouvrage *Énigmes de Tunisie* de Tahar Fazaa [1]. Ce projet a pour objectif de mettre en pratique et de tester nos compétences en HTML, CSS et JavaScript.

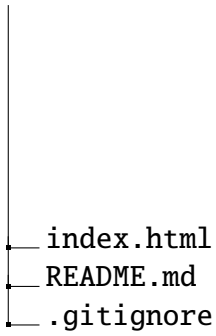
2. Cahier des charges / Objectifs

L'objectif de ce projet est de développer un site web interactif qui permet aux utilisateurs de participer à un quiz et de suivre leurs résultats en temps réel. L'interface fournit un retour visuel après chaque réponse et affiche le score final. Le score tient compte du temps écoulé, le meilleur score est sauvegardé dans `localStorage`, et une barre de progression indique l'avancement du joueur. Le site sera développé en HTML5, CSS3 et JavaScript. Il proposera une page d'accueil et une interface de quiz adaptées aux différents écrans, tout en prenant en compte l'accessibilité, en intégrant des animations CSS et en respectant les standards du W3C.

3. Conception

Structure du dossier :

```
Tchanchinet/
├── assets/
│   └── tunisian_riddles_40.json
├── css/
│   ├── styles.css
│   ├── index.css
│   ├── quiz-shared.css
│   ├── quiz.css
│   ├── results.css
│   ├── versus.css
│   ├── versus-quiz.css
│   └── versus-results.css
├── js/
│   ├── quiz-shared.js
│   ├── quiz-transition.js
│   ├── scores.js
│   ├── results.js
│   ├── quiz.js
│   ├── versus-quiz.js
│   └── versus-results.js
└── pages/
    ├── quiz.html
    ├── results.html
    ├── versus.html
    ├── versus-quiz.html
    └── versus-results.html
```



Le projet est organisé en plusieurs dossiers afin de séparer les différents composants du site et de faciliter sa maintenance.

- **assets/** : contient les ressources utilisées dans le site, notamment les différentes images et icônes, ainsi que les devinettes, leurs indices et leurs réponses.
- **css/** : regroupe tous les fichiers CSS.
- **js/** : regroupe les scripts JavaScript :
  - `quiz-shared.js` : règles communes aux deux modes (chargement des questions, vérification des réponses, tentatives, indice, pénalités temporelles et retours visuels) ;
  - `quiz.js` : mode solo avec le compte à rebours ;
  - `versus-quiz.js` : mode défi et alternance des joueurs ;
  - `quiz-transition.js` : animation entre les questions ;
  - `scores.js` : sauvegarde et affichage des meilleurs scores ;
  - `results.js`, `versus-results.js` : pages de résultats.
- **pages/** : contient les pages HTML du quiz et des résultats (mode solo et versus).
- **index.html** : représente la page d'accueil du site.

Parcours de navigation

Le site propose deux parcours, depuis la page d'accueil :

- **Mode individuel :** `index.html` → `quiz.html?mode=solo` → `results.html`.
- **Mode défi :** `index.html` → `versus.html` → `versus-quiz.html` → `versus-results.html`.

Structure HTML

Les pages du site sont développées en HTML5 et suivent une structure organisée :

- `header` : contient le logo du site sur toutes les pages. Dans les interfaces de quiz, il affiche également les informations du jeu, comme le score, le temps restant ou le nombre de tentatives.
- `nav` : permet de naviguer entre les différentes pages du site, notamment de revenir à la page d'accueil, ainsi que de choisir entre les différents modes de jeu.
- `main` : regroupe le contenu principal de chaque page.
- `section` : permet d'organiser le contenu en différentes parties, comme les questions, les indices et les scores des joueurs.
- `form` : permet la saisie des noms des joueurs dans le mode versus.

Nous n'avons pas utilisé de `footer`, car nous n'en avions pas besoin dans notre site.

Charte graphique

Pour le design du site, nous avons choisi de nous inspirer du style traditionnel tunisien afin de créer une ambiance qui rappelle notre culture et notre patrimoine.

- `Thème visuel` : nous avons utilisé une image de portes tunisiennes comme arrière-plan du site. Nous avons également ajouté des éléments traditionnels comme la chéchia, la khomsa et le jasmin.

- `Couleurs` : nous avons principalement choisi le bleu et le blanc, qui créent une ambiance traditionnelle et chaleureuse.
- `Polices` : nous avons utilisé des polices simples et lisibles adaptées à l'affichage du texte en arabe.

Outils utilisés

- `Visual Studio Code` : pour écrire et organiser le code du projet.
- `Microsoft Edge` : pour tester et visualiser les différentes pages du site.
- `Live Server` : pour lancer le site en local afin d'effectuer les tests avec Lighthouse.
- `Google Gemini` : pour générer certaines images utilisées dans le design du site.

4. Implémentation (Code et explications)

4.1 Structure de la page et interface en arabe

Les pages du jeu déclarent l'arabe comme langue et utilisent une direction droite-à-gauche. La structure HTML s'appuie sur des éléments sémantiques et intègre des aides à l'accessibilité, comme le lien d'évitement et le label associé au champ de réponse.

```
<html lang="ar" dir="rtl">
<body class="quiz-page">
  <a class="skip-link" href="#main">...</a>
  <div id="quiz" class="game-screen"
      data-attempts-left="3"
      data-status="ready"
      data-hint-state="hidden">
    <header class="game-header">...</header>
    <main id="main" class="quiz-main">
      <section class="ornament-panel question-panel">
        <h1 id="question-text">...</h1>
      </section>
      <label class="visually-hidden" for="answer">...</label>
      <input id="answer" type="text" dir="rtl"
          autocomplete="off">
      <button id="check-answer" type="button">...</button>
    </main>
  </div>
</body>
```

Les attributs `lang="ar"` et `dir="rtl"` définissent la langue et le sens de lecture. Le lien d'évitement permet d'accéder directement au contenu principal. Le label, masqué visuellement, reste accessible aux lecteurs d'écran.

4.2 Formulaire de saisie des noms (mode deux joueurs)

Le formulaire de préparation du défi valide les deux noms en HTML et les transmet à la page du quiz avec la méthode GET.

```
<form id="players-form" action="versus-quiz.html"
    method="get">
 <input type="hidden" name="mode" value="versus">

 <label for="player-one-name">...</label>
 <input id="player-one-name" name="player1"
       type="text" maxlength="24"
       pattern=".*\S.*" required>

 <label for="player-two-name">...</label>
 <input id="player-two-name" name="player2"
       type="text" maxlength="24"
       pattern=".*\S.*" required>

 <button type="submit">...</button>
</form>
```

L'attribut `required` rend les deux champs obligatoires, `pattern` efuse un nom composé uniquement d'espaces et `maxlength` limite sa longueur à 24 caractères. Le script du quiz relit les paramètres de l'URL, puis vérifie et limite à nouveau les noms

4.3 Gestion des états avec les attributs data-*

Le JavaScript met à jour les attributs de #quiz ; le CSS utilise ces états pour afficher les retours visuels, les tentatives restantes et l'indice.

```
[data-attempts-left="2"] [data-attempt="1"]
.chechia-red {
 display: none;
}

[data-attempts-left="2"] [data-attempt="1"]
.chechia-gray {
 display: block;
}

[data-status="wrong"] .feedback-wrong {
 display: block;
}

[data-status="correct"] .feedback-correct {
 display: block;
}

[data-status="exhausted"] .feedback-exhausted {
 display: block;
}
```

```
[data-attempts-left="1"]
[data-hint-state="revealed"] .hint-content {
  display: block;
}
```

Le JavaScript modifie ces attributs selon les actions du joueur :

```
quiz.dataset.attemptsLeft = String(attemptsLeft);
quiz.dataset.status = "wrong";
quiz.dataset.hintState = "revealed";
```

La séparation entre logique et présentation facilite la maintenance : le script modifie l'état du jeu, tandis que les règles CSS choisissent les éléments visibles. Après deux erreurs, il reste une tentative et l'indice peut être débloqué ; il n'apparaît qu'après activation de la khomsa.

4.4 Couleurs des joueurs et responsive

Les deux joueurs partagent les mêmes règles de présentation. Des variables CSS définissent leurs couleurs respectives ; des unités adaptatives et des media queries permettent d'ajuster l'interface aux différentes tailles d'écran.

```
.player-blue {
  --player-color: #3e79aa;
  --player-text: #244b7a;
  --player-background: #e4f0fa;
}

.player-gold {
  --player-color: #bf962f;
  --player-text: #795516;
  --player-background: #faf0d6;
}

.player-card {
  border: 1px solid var(--player-color);
  border-radius: 12px;
  background: var(--player-background);
  color: var(--player-text);
}
```

La classe `player-card` est commune aux deux joueurs. Seules les variables de couleur changent.

4.5 Adaptation aux différentes tailles d'écran

Pour rendre le site responsive, nous utilisons des media queries et des tailles adaptatives. Cela permet d'ajuster la disposition et la taille des éléments selon les dimensions de l'écran.

```
.game-screen {
  width: 100%;
  max-width: 1380px;
}

@media (max-width: 1100px) {
  .game-screen {
    max-width: 1000px;
    padding: 8px 12px;
  }
}
```

Les media queries permettent de modifier la disposition des éléments sur les écrans plus petits, tandis que les propriétés `width` et `max-width` permettent d'adapter leur largeur.

4.6 Logique du jeu en JavaScript

Les questions du jeu sont chargées avec `fetch` depuis un fichier JSON contenant 70 devinettes tunisiennes : 40 questions à réponse écrite et 30 questions à choix multiples (QCM). Chaque devinette possède un type, une question, un indice et la réponse attendue (une liste de réponses acceptées pour les questions écrites, une seule réponse parmi quatre options pour les QCM).

```
const response = await fetch("../assets/tunisian_riddles_40.json");
if (!response.ok) throw new Error("Failed to load riddles.");
questions = await response.json();
```

Si le chargement échoue, un message est affiché à la place de la question et les boutons sont désactivés.
Les règles communes aux deux modes (vérification, tentatives, indice, points) sont regroupées dans `quiz-shared.js`. Les fichiers `quiz.js` et `versus-quiz.js` ajoutent seulement ce qui est propre à chaque mode.
Avant de comparer la réponse du joueur avec celles du fichier JSON, nous utilisons une fonction pour supprimer les espaces inutiles.

```
function normalizeAnswer(value) {
  return value.trim().replace(/\s+/g, " ");
}
```

Cette fonction supprime les espaces au début et à la fin du texte, ainsi que les espaces répétés entre les mots.
Chaque question donne au joueur trois tentatives. Le nombre de points dépend de la tentative utilisée.

```
const POINTS_BY_ATTEMPT = [100, 80, 50];
const POINTS_WITH_HINT = 25;
const MAX_ATTEMPTS = 3;
```

Une bonne réponse rapporte 100 points à la première tentative, 80 à la deuxième et 50 à la troisième. Si le joueur utilise l'indice, il obtient seulement 25 points en cas de bonne réponse. Une pénalité de temps est ensuite retirée de ces points (voir 4.7).
Après trois mauvaises réponses, la réponse correcte est affichée. Le joueur peut également passer une question sans gagner de points.
Dans le mode solo, le joueur dispose de trois minutes pour répondre au maximum de questions possible. Les questions sont affichées dans un ordre aléatoire.

```
const TOTAL_TIME = 3 * 60;

timerInterval = setInterval(() => {
  // pause the timer while reading the result or switching questions
  if (!nextButton.disabled || questionSliding) return;
  timer--;
  timerDisplay.textContent = formatTime(timer);
  updateProgress(TOTAL_TIME - timer);
  if (timer <= 0) finishGame("time");
}, 1000);
```

Le temps restant est mis à jour chaque seconde, au format `mm:ss` grâce à la fonction `formatTime`. Le compte à rebours se met en pause lorsque le joueur lit le résultat d'une question (le bouton « question suivante » est alors actif) ou pendant l'animation de transition : le joueur ne perd donc pas de temps quand il ne peut pas répondre. La partie se termine lorsque le temps est écoulé ou lorsque toutes les questions ont été jouées.
Les joueurs alternent après chaque question terminée ou passée. Chacun répond à cinq questions ; une bonne réponse rapporte 100, 80 ou 50 points selon la tentative, et seulement 25 points si l'indice a été utilisé.

```
setActivePlayer(turnsPlayed % 2 === 0 ? 1 : 2);

sessionStorage.setItem("versusResult", JSON.stringify({
  player1: { name: players[1].name, score: players[1].score },
  player2: { name: players[2].name, score: players[2].score },
}));
window.location.href = "versus-results.html";
```

Les scores et les noms sont conservés dans `sessionStorage` jusqu'à l'affichage des résultats. La page de résultats compare les deux scores et met à jour l'attribut `data-winner` avec `1`, `2` ou `draw`.

4.7 Chronomètre caché par question et pénalités temporelles

En plus du compte à rebours général du mode solo, chaque question possède sa propre horloge, invisible pour le joueur. Elle ne s'affiche pas à l'écran : elle sert uniquement à mesurer le temps mis pour répondre à la question en cours, afin de retirer des points si le joueur est lent.
Au moment où une question est affichée, la fonction `showQuestion` enregistre l'heure de départ avec `Date.now()`, qui renvoie le nombre de millisecondes écoulées depuis 1970 :

```
function showQuestion(question) {
  currentQuestion = question;
  questionStartTime = Date.now();
  attemptsLeft = MAX_ATTEMPTS;
  hintUsed = false;
  ...
}
```

Comme `questionStartTime` est réinitialisée à chaque nouvelle question, chaque devinette repart de zéro : le temps passé sur les questions précédentes n'a aucun effet sur la question actuelle. Nous n'utilisons pas de `setInterval` pour cette horloge : il suffit de calculer la différence entre l'heure actuelle et l'heure de départ au moment où le joueur valide une bonne réponse.

```
const TIME_PENALTY_INTERVAL = 10;
const TIME_PENALTY_POINTS = 5;
const MAX_TIME_PENALTY = 25;

function calculateTimePenalty() {
  const seconds =
    Math.floor((Date.now() - questionStartTime) / 1000);
  return Math.min(
    Math.floor(seconds / TIME_PENALTY_INTERVAL)
      * TIME_PENALTY_POINTS,
    MAX_TIME_PENALTY
  );
}
```

La différence est convertie en secondes, puis divisée par 10 : le joueur perd 5 points pour chaque tranche complète de 10 secondes. `Math.min` limite la pénalité à 25 points, pour qu'une réponse lente mais juste rapporte toujours des points (sauf avec l'indice).
La pénalité est retirée des points de base au moment de la bonne réponse, et `Math.max` empêche le score de devenir négatif :

```
const basePoints = hintUsed
  ? POINTS_WITH_HINT
  : POINTS_BY_ATTEMPT[MAX_ATTEMPTS - attemptsLeft];
addPoints(Math.max(0, basePoints - calculateTimePenalty()));
```

Exemples :

- bonne réponse à la première tentative en 8 secondes : $100 - 0 = 100$ points ;
- bonne réponse à la deuxième tentative en 23 secondes : $80 - 10 = 70$ points ;
- bonne réponse avec l'indice après une minute : $25 - 25 = 0$ point.

Cette horloge est définie dans `quiz-shared.js`, elle fonctionne donc aussi en mode défi : même sans compte à rebours visible, chaque joueur est pénalisé s'il met trop de temps à répondre à sa

question. La fonction `addPoints` ajoute ensuite les points au score solo ou au joueur dont c'est le tour.

4.8 Feedback immédiat après chaque réponse

Après chaque réponse, la fonction `checkAnswer` met à jour l'attribut `data-status` de `#quiz` : `wrong` pour une mauvaise réponse, `correct` pour une bonne réponse et `exhausted` lorsque les tentatives sont épuisées. Le CSS (voir 4.3) affiche alors le message correspondant en rouge ou en vert, grise une chéchia et, si nécessaire, affiche la bonne réponse.

```
attemptsLeft = currentQuestion.type === "qcm"
  ? 0 : attemptsLeft - 1;
answerInput.setAttribute("aria-invalid", "true");
quiz.dataset.status = "wrong";
updateAttempts();
if (attemptsLeft === 1) hintToggle.disabled = false;
if (attemptsLeft === 0) {
  endCurrentQuestion("exhausted");
  return;
}
```

La zone de message utilise `role="status"` et `aria-live="polite"` : le résultat est donc aussi annoncé par les lecteurs d'écran. Lorsque la question est terminée, `endCurrentQuestion` désactive le champ et les boutons de réponse, pour qu'une même question ne puisse pas rapporter des points deux fois.

4.9 Sauvegarde des meilleurs scores (localStorage)

Les meilleurs scores sont gérés dans `scores.js`. Ils sont enregistrés dans `localStorage`, qui conserve les données même après la fermeture du navigateur. Chaque mode possède sa propre clé : `soloBestScores` et `versusBestScores`.

```
function readTopScores(key) {
  return JSON.parse(localStorage.getItem(key) || "[]");
}

function saveTopScores(key, entries) {
  const scores = [...readTopScores(key), ...entries];
  scores.sort((a, b) => b.score - a.score);
  localStorage.setItem(key, JSON.stringify(scores.slice(0, 5)));
}
```

`localStorage` ne stocke que du texte : les scores sont donc convertis avec `JSON.stringify` et relus avec `JSON.parse`. Les nouveaux scores sont ajoutés aux anciens, triés du plus grand au plus petit, puis seuls les cinq meilleurs sont gardés. En mode défi, le nom de chaque joueur est enregistré avec son score.
À la fin d'une partie, `finishGame` appelle `saveTopScores` une seule fois (grâce à la variable `gameOver`), puis la page de résultats affiche le classement avec `renderLeaderboard`. Les

lignes sont créées avec `createElement` et `textContent`, ce qui évite d'interpréter un nom de joueur comme du HTML.

4.10 Types de questions : réponse écrite et QCM

Le champ `type` de chaque devinette indique son type. La fonction `showQuestion` copie ce type dans `data-type`, et le CSS affiche soit le champ de saisie, soit les quatre boutons de choix.

```
quiz.dataset.type = question.type;

for (let i = 0; i < optionButtons.length; i++) {
  const button = optionButtons[i];
  button.textContent = "";
  if (question.options && question.options[i]) {
    button.textContent = question.options[i];
  }
}
```

Un clic sur une option recopie son texte dans le champ de réponse et appelle la même fonction `checkAnswer` que pour une réponse écrite : il n'y a donc qu'une seule logique de vérification. Un QCM n'accepte qu'un seul choix et n'a pas d'indice. À la fin, la bonne option est colorée en vert et le mauvais choix en rouge grâce à l'attribut `data-result`. Les flèches du clavier permettent de se déplacer entre les options, en respectant le sens droite-à-gauche.

4.11 Barre de progression visuelle

La barre suit le temps écoulé en mode solo et le nombre de questions jouées en mode défi. Un élément `progress` garde la valeur pour l'accessibilité, et un `span` décoratif est élargi en pourcentage. Sa couleur change selon ce qui reste.

```
function updateProgress(value) {
  progressBar.value = value;
  const percentage = (progressBar.value / progressBar.max) * 100;
  progressFill.style.width = percentage + "%";
  const remaining = progressBar.max - value;
  const redLimit = progressBar.max / 6;
  const yellowLimit = progressBar.max / 3;

  if (remaining <= redLimit) {
    progressBar.dataset.level = "danger";
  } else if (remaining <= yellowLimit) {
    progressBar.dataset.level = "warning";
  } else {
    progressBar.dataset.level = "normal";
  }
}
```

La barre est bleue au début, devient jaune quand il reste un tiers du temps (ou des questions), puis rouge pour le dernier sixième.

4.12 Animation de transition entre les questions

Lors du passage à la question suivante, `quiz-transition.js` fait sortir l'ancienne question vers la gauche, change son contenu, puis fait entrer la nouvelle depuis la droite. L'attribut `data-transition` déclenche les animations CSS.

```
function slideQuestion(changeQuestion) {
 if (questionSliding) return;
 const reducedMotion = window
  .matchMedia("(prefers-reduced-motion: reduce)").matches;
 if (reducedMotion) {
  changeQuestion();
  return;
 }
 questionSliding = true;
 main.inert = true;
 quizBox.dataset.transition = "out";
 setTimeout(function () {
  changeQuestion();
  quizBox.dataset.transition = "in";
  setTimeout(finishQuestionSlide, 150);
 }, 150);
}
```

Pendant l'animation, `main.inert` bloque les clics pour éviter de sauter deux questions d'un coup, et le compte à rebours est mis en pause. Si l'utilisateur a activé la réduction des animations (`prefers-reduced-motion`), la question change directement, sans animation.

5. Résultat final

Le site propose un parcours complet autour des devinettes tunisiennes. La page d'accueil présente les règles et donne accès au jeu individuel ou au défi à deux. En mode individuel, le joueur répond aux questions avant la fin du compte à rebours ; en mode défi, les deux joueurs jouent à tour de rôle sur le même appareil. Une barre de progression suit le temps restant en mode individuel et l'avancement des questions en mode défi. Les questions alternent entre réponses écrites et QCM, avec une courte animation entre chaque question. À la fin de chaque mode, un classement distinct des cinq meilleurs scores, sauvegardé dans `localStorage`, présente les résultats des joueurs. Les captures suivantes illustrent les principales étapes du jeu.
L'interface en arabe, orientée de droite à gauche, reprend des motifs traditionnels dans une palette dominée par le bleu et le blanc. Les indices, les tentatives et les scores disposent de retours visuels, tandis que les couleurs bleu et or distinguent les joueurs du mode défi.



Page d'accueil : règles et choix du mode de jeu.



Début du quiz individuel : énigme, temps, tentatives et progression liée au compte à rebours.



Retour visuel après une première réponse incorrecte.



Après deux erreurs, l'indice est débloqué et peut être consulté.



Affichage de l'indice sous la question.



Réponse correcte et mise à jour du score après utilisation de l'indice.



Poursuite du quiz individuel : progression liée au temps et score actualisé.



Résultats du mode solo : score final et classement individuel.



Formulaire de saisie des noms avant le défi à deux joueurs.



Mode défi : question en cours et progression selon le nombre de questions.



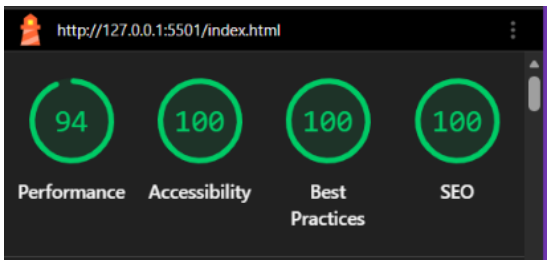
Tour suivant : la barre progresse avec les questions et les scores des deux joueurs sont affichés.

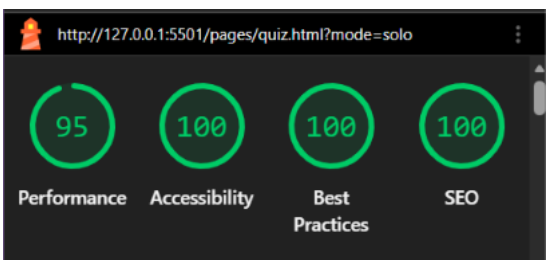


Résultats du mode défi : classement distinct, scores finaux et gagnant.

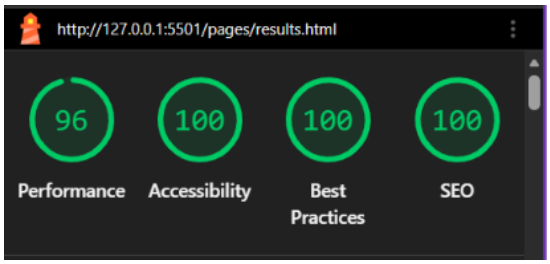
Scores Lighthouse

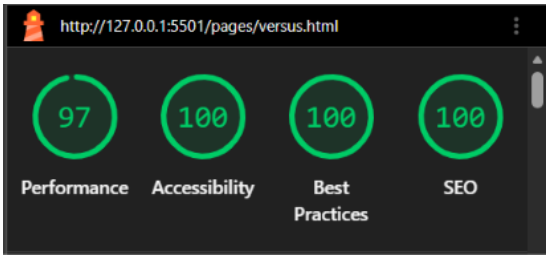
Les pages ont été testées avec Lighthouse, en lançant le site avec Live Server. Toutes obtiennent 100 en accessibilité, bonnes pratiques et SEO, et entre 94 et 97 en performance.



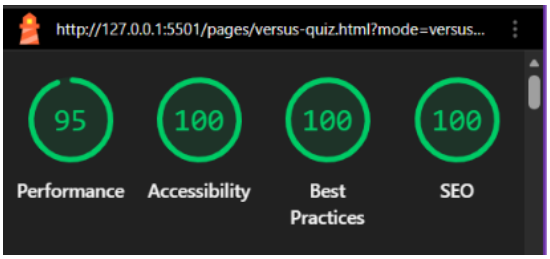


Page d'accueil (à gauche) et quiz en mode solo (à droite).





Résultats du mode solo (à gauche) et saisie des joueurs (à droite).



Quiz en mode défi.

Dépôt GitHub : `https://github.com/Nermine-Galai/Tchanchinet`

6. Difficultés rencontrées et solutions

- **Comparaison des réponses en arabe :** une même réponse pouvait être écrite de plusieurs façons. Pour résoudre ce problème, nous avons ajouté une liste de réponses acceptées pour chaque devinette dans le fichier JSON. Une fonction de normalisation permet également de supprimer les espaces inutiles avant la comparaison.
- **Gestion des états du quiz :** il fallait réinitialiser les tentatives, l'indice et les commandes à chaque nouvelle énigme, tout en empêchant une réponse ou un score d'être enregistré plusieurs fois. Les scripts suivent l'état de la question et désactivent les commandes inappropriées après sa résolution ou son abandon.
- **Équilibre du mode défi :** les tours et les points devaient être suivis séparément pour chaque participant. Le jeu alterne les joueurs après chaque question et limite chaque joueur à cinq questions, sans compte à rebours visible ; seule l'horloge cachée de chaque question applique les pénalités de temps.
- **Pénalités temporelles justes :** il fallait éviter que le temps passé sur une question, sur l'affichage du résultat ou sur l'animation pénalise le joueur à la question suivante. Nous avons donné à chaque question sa propre horloge, remise à zéro dans `showQuestion`, et mis le compte à rebours en pause pendant ces moments. La pénalité est aussi plafonnée à 25 points.
- **Double clic pendant la transition :** un clic rapide sur « question suivante » pendant l'animation risquait de faire sauter une question. La variable `questionSliding` et l'attribut `inert` bloquent les actions jusqu'à la fin de l'animation.
- **Rejouer sans recharger la page :** relancer une partie risquait de créer un deuxième chronomètre ou réutiliser une liste de questions déjà vidée. `startGame` arrête l'ancien `setInterval` avec `clearInterval` et repart d'une copie complète de la liste.
- **Deux types de questions :** les QCM et les questions écrites devaient partager la même logique. Le clic sur une option recopie sa valeur dans le champ de réponse, puis la même fonction `checkAnswer` vérifie la réponse.

7. Conclusion

Ce projet nous a permis de consolider nos connaissances en HTML, CSS et JavaScript, notamment sur la structuration d'une page web, l'utilisation des sélecteurs CSS, la conception responsive et la gestion des interactions et de la logique du jeu avec JavaScript.
Il constitue une base solide pour aborder des projets de développement web plus avancés à l'avenir.

Références

[1] Fazaa, Tahar. *Énigmes de Tunisie*. Tunis : Sindbad éditions, 2014.

[2] MDN Web Docs. *CSS*. `https://developer.mozilla.org/en-US/docs/Web/CSS/`

[3] MDN Web Docs. *JavaScript*. https://developer.mozilla.org/en-US/docs/Web/JavaScript

[4] MDN Web Docs. *Window : localStorage property*. https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage

[5] Stack Overflow. https://stackoverflow.com